Stack

Functions

- A function is a block of code that performs specific tasks; normally, a program contains many functions.
- When a function is called, the control is transferred to a different memory address.
- The CPU then executes the code at that memory address, and it comes back (control is transferred back) after it finishes running the code.

Functions

- The function contains multiple components:
  - function can take data as input via parameters,
  - it has a body that contains the code it executes,
  - it contains local variables that are used to temporarily store values, and
  - it can output data.

Functions

- The parameters, local variables, and function flow controls are all stored in an important area of the memory called the **stack.**

Stack

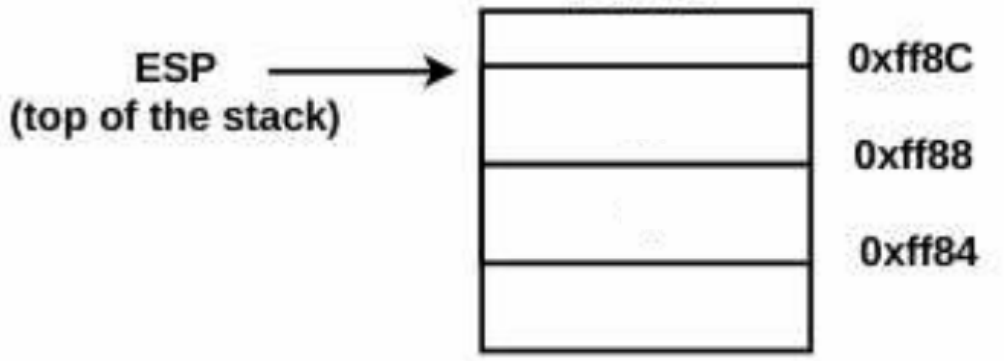
- The stack is an area of the memory that gets allocated by the operating system when the thread is created.
- The stack is organized in a Last-In-First-Out (LIFO) structure, which means that the most recent data that you put in the stack will be the first one to be removed from the stack.

Stack Operations

- You put data (called pushing) onto the stack by using the push instruction,and
- you remove data (called popping) from the stack using the pop instruction.
- The push instruction pushes a 4-byte value onto the stack
- The pop instruction pops a 4-byte value from the top of the stack.

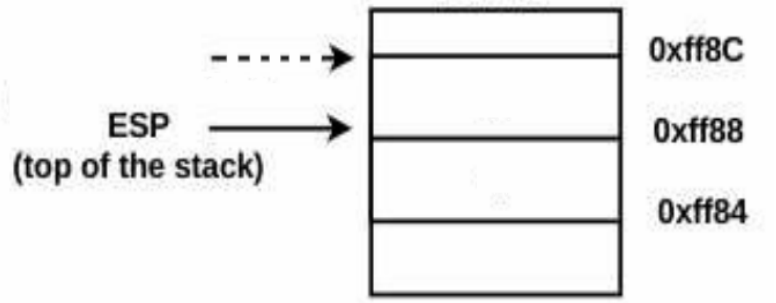
- push source
  - pushes source on top of the stack
- pop destination
  - copies value from the top of the stack to the destination

- The stack grows from higher addresses to lower addresses.
- When a stack is created, the esp register (also called the stack pointer) points to the top of the stack (higher address)
-
-



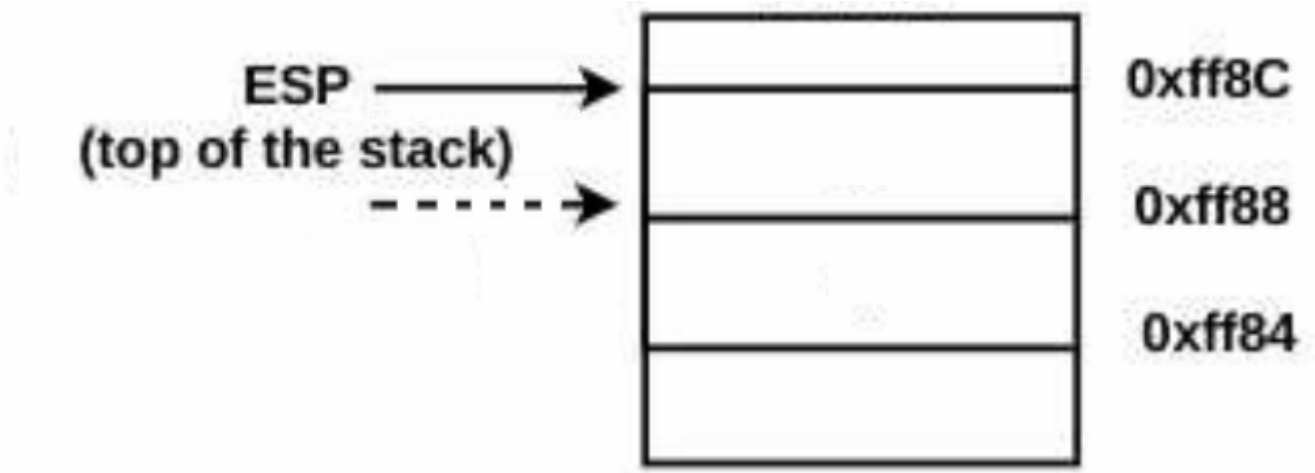
PUSH

- When you push data into the stack, the esp register decrements by 4 (esp-4) to a lower address.



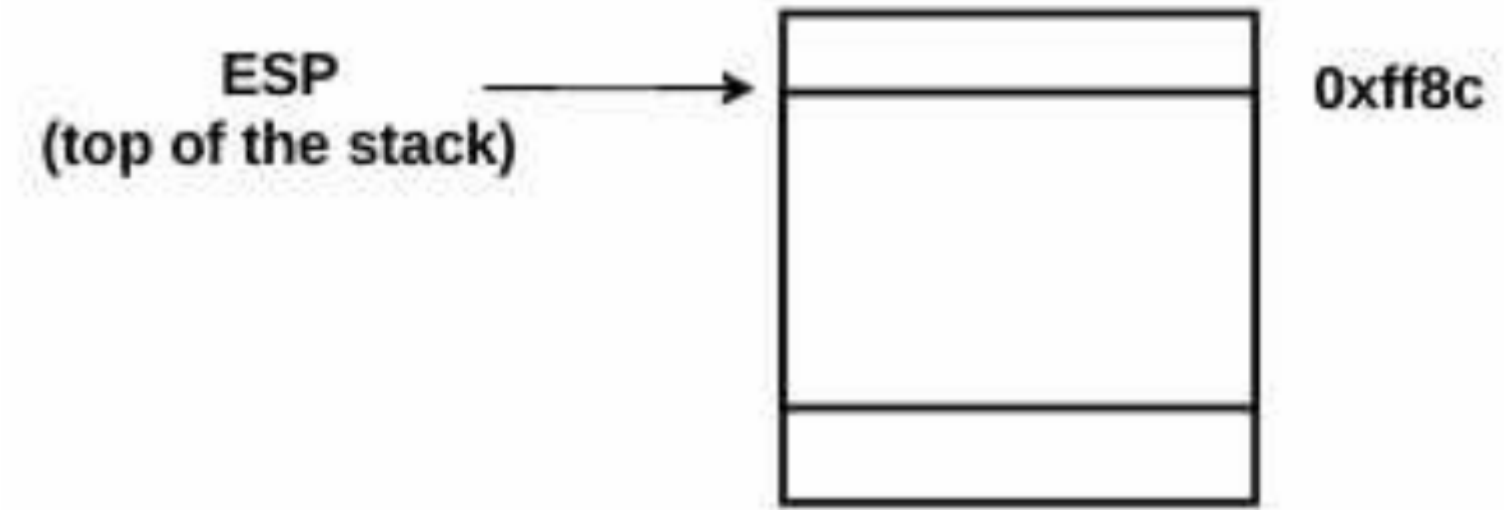
POP

- When you pop a value, the esp increments by 4 (esp+4).



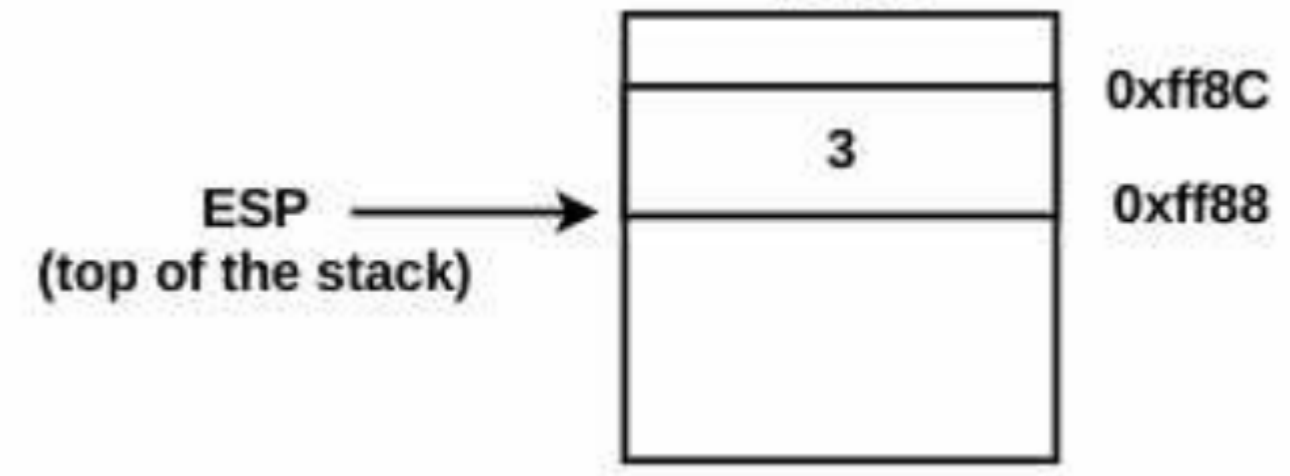
Initially

```
push 3
push 4
pop ebx
pop edx
```



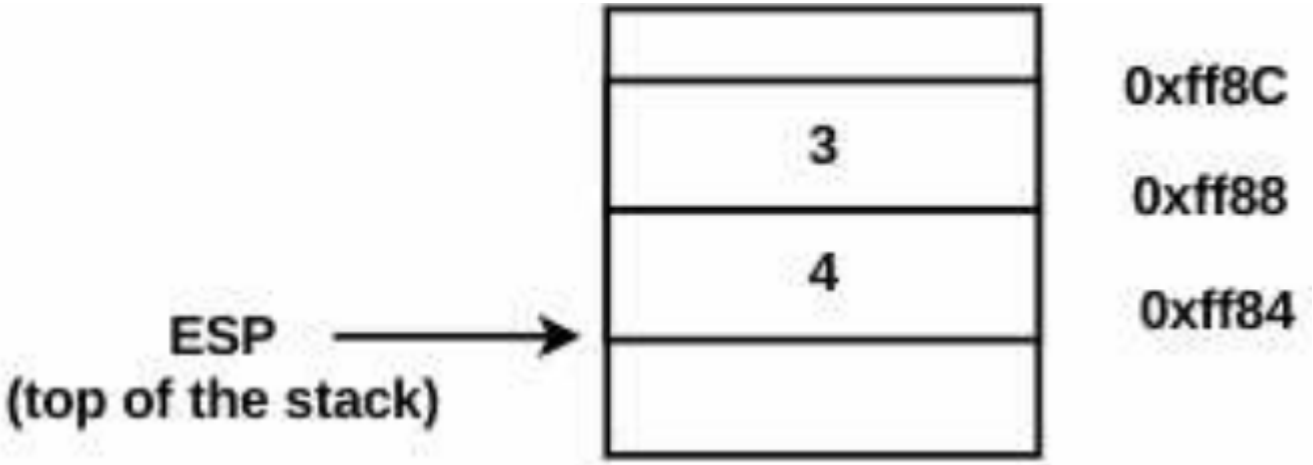
First PUSH

```
push 3
push 4
pop ebx
pop edx
```



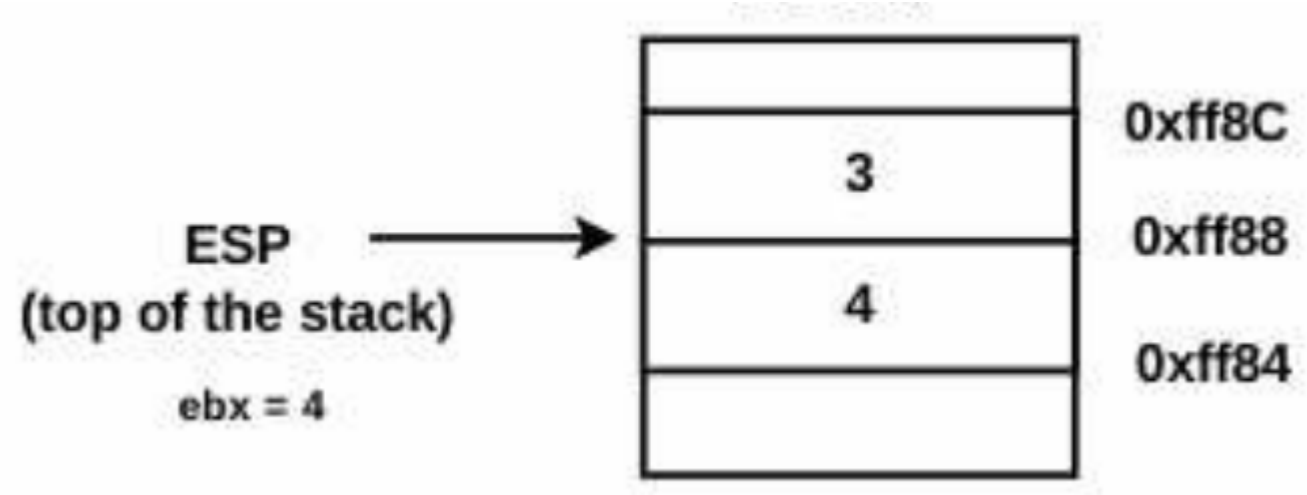
Second PUSH

```
push 3
push 4
pop ebx
pop edx
```



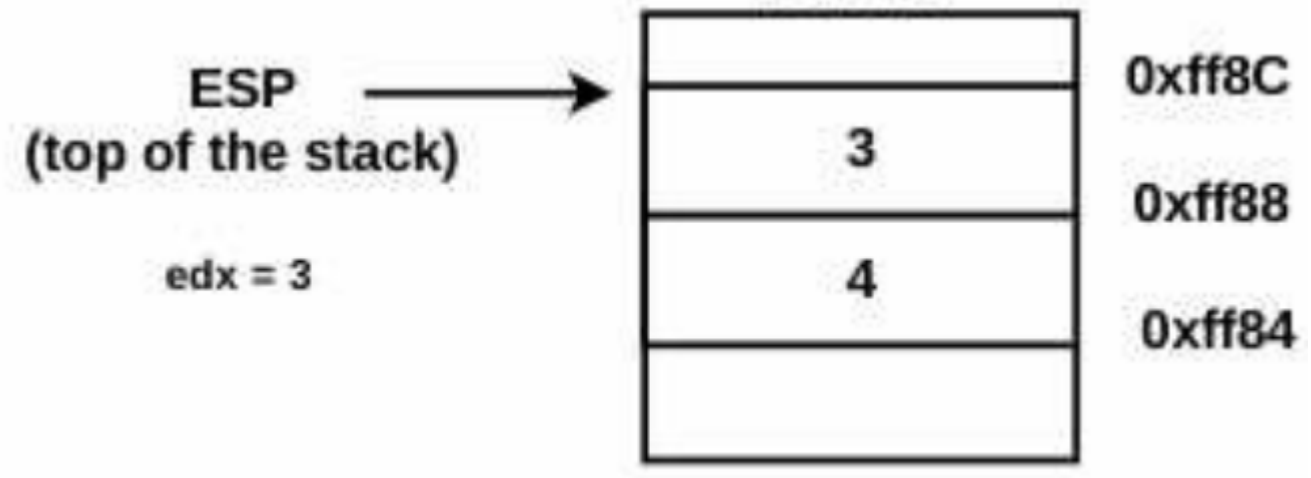
First POP

```
push 3
push 4
pop ebx
pop edx
```



Second POP

```
push 3
push 4
pop ebx
pop edx
```



Calling Functions

- The call instruction in the assembly language can be used to invoke a function.
-
```
call <some_function>
```

Role of Stack in Function Calls

`call <some_function>`

- From a code analysis perspective, think of `some_function` as an address containing a block of code.
- When the `call` instruction is executed, the control is transferred to `some_function` (a block of code), but before that, it stores the address of the next instruction (the instruction following `call <some_function>`) by pushing it onto the stack.
- The address following the `call` which is pushed onto the stack is called the *return address*.
- Once `some_function` finishes executing, the return address that was stored on the stack is popped from the stack, and the execution continues from the popped address.

Returning From Function

- In assembly language, to return from a function, you use the ret instruction.
- This instruction pops the address from the top of the stack
  - The popped address is placed in the EIP register
  - The control is transferred to the popped address.

Function Parameters And Return Values

- In the x86 architecture,
  - Parameters that a function accepts are pushed onto the stack
  - Return value is placed in the EAX register.

```c
int sum(int a, int b) {
    return a + b;
}

int main() {
    // Call sum(2, 3) and return the result (5) as exit code
    return sum(2, 3);
}
```

```asm
sum:
    push    ebp
    mov     ebp, esp
    mov     eax, [ebp+8]
    add     eax, [ebp+12]
    pop     ebp
    ret

main:
    push    3
    push    2
    call    sum
    add     esp, 8
    pop     ebp
    ret
```

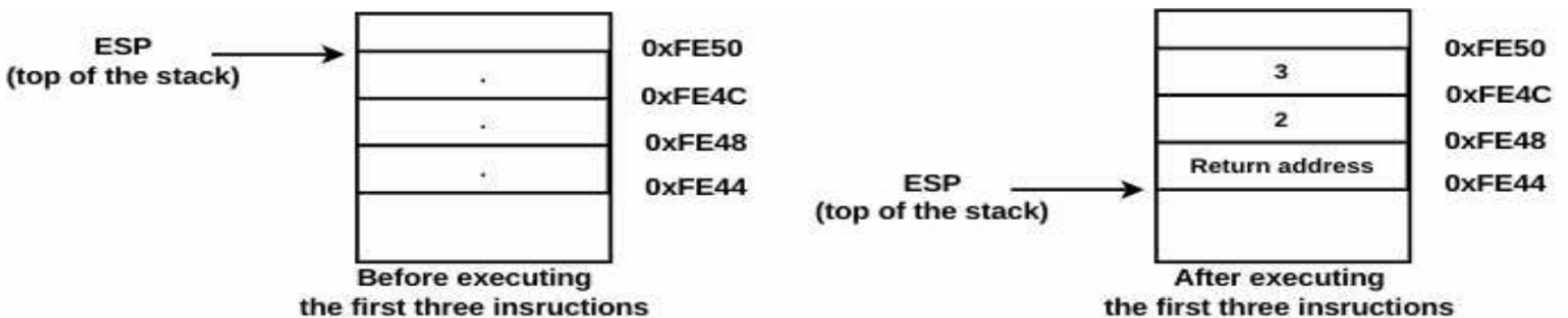
```
int test(int a, int b)
{
    int x, y;
    x = a;
    y = b;
    return 0;
}

int main()
{
   test(2, 3);
   return 0;
}
```

```
int main()
{
    test(2, 3);
    return 0;
}
```

```
push 3  ❶
push 2  ❷
call test  ❸
add esp, 8 ; after test is exectued, the control is returned here
xor eax, eax
```

```
int main()
{
    test(2, 3);
    return 0;
}
```

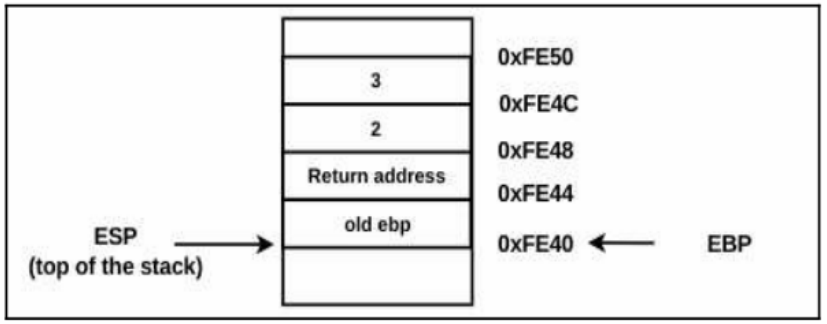
```
push 3  ❶
push 2  ❷
call test  ❸
add esp, 8 ; after test is exectued, the control is returned here
xor eax, eax
```

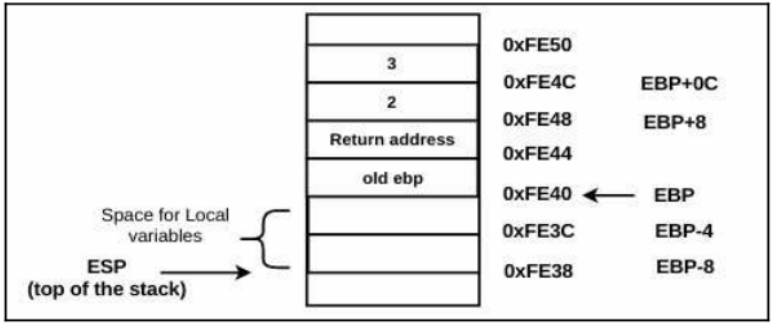


Test Function

```
int test(int a, int b)
{
    int x, y;
    x = a;
    y = b;
    return 0;
}
```

```
push ebp   ❹
mov ebp, esp   ❺
sub esp, 8   ❽
mov eax, [ebp+8]
mov [ebp-4], eax
mov ecx, [ebp+0Ch]
mov [ebp-8], ecx
xor eax, eax   ❾
mov esp, ebp   ❻
pop ebp   ❼
ret   ❿
```





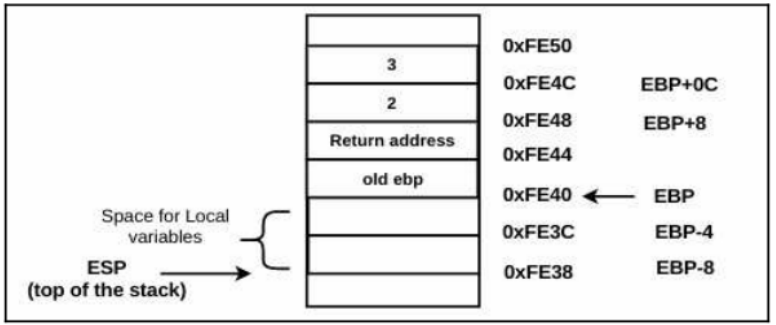
4 & 5 function prologue

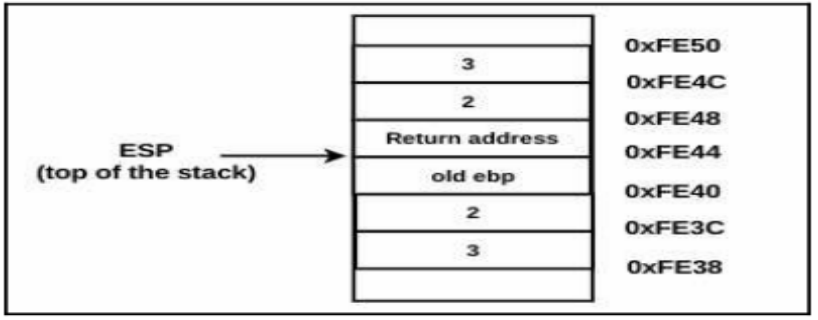
6 & 7 function epilogue

Test Function

```
int test(int a, int b)
{
    int x, y;
    x = a;
    y = b;
    return 0;
}
```

```
push ebp  ❹
mov ebp, esp  ❺
sub esp, 8  ❽
mov eax, [ebp+8]
mov [ebp-4], eax
mov ecx, [ebp+0Ch]
mov [ebp-8], ecx
xor eax, eax  ❾
mov esp, ebp  ❻
pop ebp  ❼
ret  ❿
```





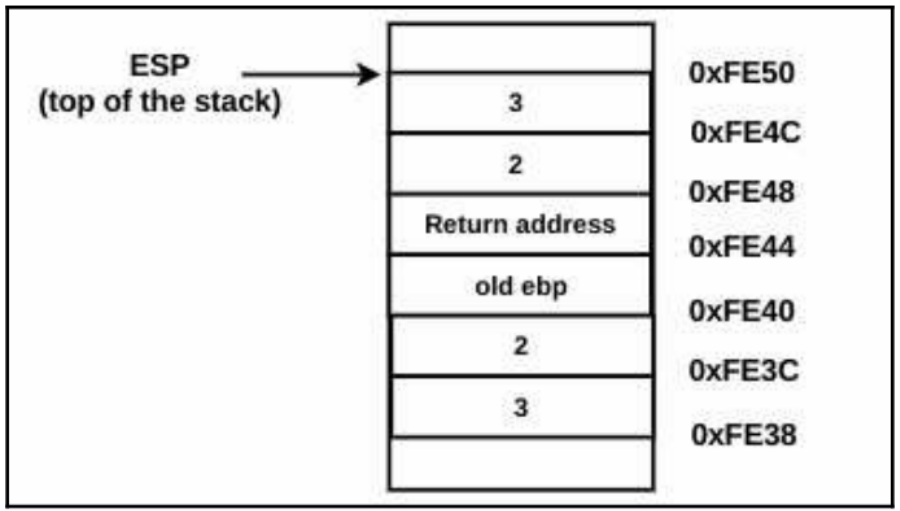

```
mov eax, [ebp+8]
mov [ebp-4], eax
mov ecx, [ebp+0Ch]
mov [ebp-8], ecx
```

```
mov eax, [a]
mov [x], eax
mov ecx, [b]
mov [y], ecx
```

```
x = a
y = b
```

4 & 5 function prologue

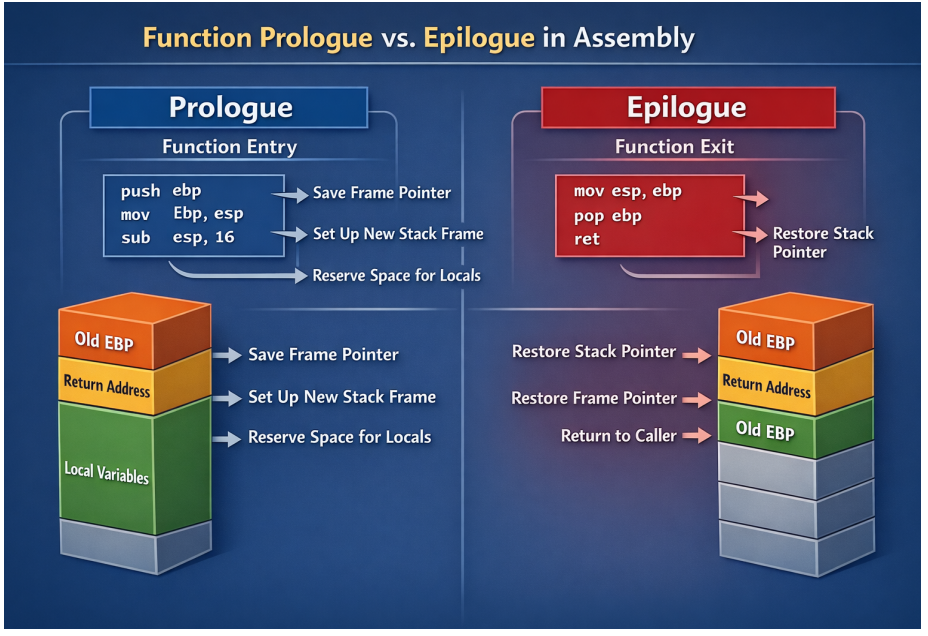
6 & 7 function epilogue



- when the ret instruction is executed, the return address on top of the stack is popped out and placed in the eip register.

Prologue Vs Epilogue

- The function prologue constructs the stack frame and preserves execution context so the function can operate without breaking the caller.
- A function epilogue is the sequence of instructions at the end of a function that:
  - Restores the stack and registers to their previous state
  - Prepares the return value (if any)
  - Returns control to the caller



Stack Cleanup

- **cdecl (C default on 32-bit x86)**
  - Parameters pushed right-to-left
  - Caller removes parameters after return
- **stdcall (Windows API)**
  - Parameters pushed right-to-left
  - Callee cleans up the stack
  - Cleanup happens inside function epilogue (e.g., ret 8)
  - Common in Microsoft DLL exports

🔥 cdecl vs stdcall (x86 32-bit)

Feature	cdecl	stdcall
Who pushes arguments	Caller	Caller
Order	Right → Left	Right → Left
Who cleans stack	**Caller**	**Callee**
Where cleanup occurs	After `call`	In epilogue
Typical cleanup	`add esp, N`	`ret N`
Used by	C programs	Windows APIs

Call, Prologue, Epilogue: Who Does What?

Action	Who Performs It
Push arguments	Caller
‘call’ instruction	Caller
✓ Prologue	✓ Callee
Function body	Callee
✓ Epilogue	✓ Callee
Argument cleanup (cdecl)	Caller
Argument cleanup (stdcall)	Callee epilogue